VideoAnalyzer — Project Specification & Developer Blueprint

> **Version:** 1.0.0
> **Author:** S. Pratham
> **Status:** Pre-development (Specification Phase)

Table of Contents

1. [Project Overview](#1-project-overview)
2. [System Architecture](#2-system-architecture)
3. [Directory Structure](#3-directory-structure)
4. [Pipeline — Step by Step](#4-pipeline--step-by-step)
   - 4.1 Input Layer
   - 4.2 Audio Extraction & Transcription
   - 4.3 Frame Extraction (Scene-Change Based)
   - 4.4 Frame Analysis with Qwen3-VL
   - 4.5 Master Summary Generation
   - 4.6 Prompt Q&A Layer (Optional)
   - 4.7 Output & Cleanup
5. [Models & Dependencies](#5-models--dependencies)
6. [Hardware Detection & Optimization](#6-hardware-detection--optimization)
7. [Module API (Importable Interface)](#7-module-api-importable-interface)
8. [Logging Standard](#8-logging-standard)
9. [Configuration Reference](#9-configuration-reference)
10. [Frame Extraction Module (Reference Code)](#10-frame-extraction-module-reference-code)
11. [Development Guidelines](#11-development-guidelines)
12. [Error Handling Strategy](#12-error-handling-strategy)

---

1. Project Overview

VideoAnalyzer is a local, fully offline AI pipeline that takes a video file as input and produces an extraordinarily detailed, multi-page natural language analysis of its content — covering both the spoken audio and the visual frames.

It is designed to be:
- **Importable** as a Python module into any project
- **Device-independent**, running on CPU or GPU with automatic detection and optimization
- **Self-sufficient**, detecting and downloading required models on first run only
- **Heavily logged**, with every stage printing structured status output
- **Ephemeral by default**, with all temp files auto-deleted after a run

What it does in plain English

You point it at a video file. Optionally, you give it a question or instruction (a "prompt"). It extracts the audio, transcribes every word spoken, extracts key visual frames based on scene changes, describes every frame using a vision-language model, feeds everything into a large language model, and returns either a dense multi-page summary — or a precise, evidence-backed answer to your question.

Example use cases

- "What is being taught in this lecture video?"
- "List every product shown in this advertisement."
- "Summarize the key arguments made in this interview."
- "What happens at the 2-minute mark?"
- Running with no prompt → returns a comprehensive standalone summary document.

---

2. System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                          INPUT LAYER                         │
│   video_path: str  +  prompt: str | None                     │
└──────────────────────────┬───────────────────────────────────┘
                           │
            ┌──────────────┴──────────────┐
            │                             │
            ▼                             ▼
┌──────────────────────┐        ┌──────────────────────────┐
│   AUDIO PIPELINE     │        │      VIDEO PIPELINE      │
│                      │        │                          │
│  1. Extract audio    │        │  1. Scene-change frame   │
│     (ffmpeg)         │        │     extraction (OpenCV + │
│                      │        │     SSIM + Histogram)    │
│  2. Transcribe with  │        │                          │
│     distil-large-v3  │        │  2. Frame analysis with  │
│     (faster-whisper) │        │     qwen3-vl:2b (Ollama) │
│                      │        │     → one description    │
│  → transcription.txt │        │       per keyframe       │
│     (temp)           │        │                          │
└──────────┬───────────┘        │  → frame_analyses.txt (temp) │
           │                    └──────────┬───────────────┘
           └──────────────┬────────────────┘
                          │
                          ▼
```

```
            ┌──────────────────────────┐
            |   SYNTHESIS LAYER          |
            |                            |
            |   LLM: qwen3.5:9b (Ollama)  |
            |                            |
            |   Input:                   |
            |   - Full transcription      |
            |   - All frame descriptions  |
            |                            |
            |   Output:                  |
            |   → Master Summary (2-3+    |
            |     pages, ultra-detailed)  |
            └──────────┬───────────────┘
                       |
              ┌────────┴────────┐
              |                 |
        prompt=None           prompt=str
              |                 |
              ▼                 ▼
      ┌──────────────┐   ┌──────────────────────┐
      |  Return summary |   |  Q&A LAYER              |
      |  to caller      |   |                        |
      └──────────────┘   |  LLM: qwen3.5:9b        |
                         |  Input: summary + prompt  |
                         |                        |
                         |  Output:                |
                         |  - Direct answer        |
                         |  - Relevant summary     |
                         |    sections cited       |
                         └──────────────────────┘
```

---

3. Directory Structure

```
VideoAnalyzer/
|
├── video_analyzer/                  ← Main package (importable)
|   ├── __init__.py                  ← Public API: analyze()
|   ├── analyzer.py                  ← Top-level pipeline orchestrator
|   ├── audio/
|   |   ├── __init__.py
|   |   ├── extractor.py             ← ffmpeg audio extraction
```

```
|   |   └── transcriber.py          ← distil-large-v3 transcription
|   ├── video/
|   |   ├── __init__.py
|   |   ├── frame_extractor.py      ← Scene-change keyframe extraction
|   |   └── frame_analyzer.py       ← qwen3-vl:2b frame description
|   ├── llm/
|   |   ├── __init__.py
|   |   ├── ollama_manager.py       ← Ollama process + model management
|   |   └── summarizer.py           ← Summary + Q&A prompting logic
|   ├── utils/
|   |   ├── __init__.py
|   |   ├── hardware.py             ← GPU/CPU detection & config
|   |   ├── logger.py               ← Centralized logging setup
|   |   ├── model_manager.py        ← Model download/verification
|   |   └── cleanup.py              ← Temp file lifecycle management
|   └── config.py                   ← All tunable defaults
|
├── ingestion/                      ← Drop your video files here
|   └── video.mp4                   ← (example)
|
├── artifacts/                      ← Runtime artifacts
|   └── video_frames/               ← Keyframes saved during run
|
├── requirements.txt
├── setup.py
└── README.md
```

---

4. Pipeline — Step by Step

4.1 Input Layer

File: `video_analyzer/analyzer.py` → `analyze()`

The entry point accepts:

```python
analyze(
    video_path: str,          # Absolute or relative path to video file
    prompt: str | None = None # Optional user question / instruction
) -> AnalysisResult
```

On entry, the pipeline must:

1. Log the start timestamp, video path, and detected file size.
2. Validate that the video file exists and is readable. Raise `FileNotFoundError` with a clear message if not.
3. Detect hardware (see §6) and log the result.
4. Verify all required models and tools are available (see §5). Download anything missing.
5. Ensure Ollama is running; start it if not (see §5.3).
6. Create a unique temporary working directory for this run (e.g., `/tmp/videoanalyzer_{uuid4}/`).

4.2 Audio Extraction & Transcription

Files: `audio/extractor.py`, `audio/transcriber.py`

Step 1 — Extract audio with ffmpeg

Use `ffmpeg` (via `subprocess`) to strip the audio track from the video and save it as a `.wav` file into the temp directory.

```
LOG: [AUDIO] Extracting audio from video...
LOG: [AUDIO] Source: /path/to/video.mp4 (duration: Xs, size: XMB)
LOG: [AUDIO] Writing audio → /tmp/videoanalyzer_<uuid>/audio.wav
LOG: [AUDIO] ✓ Audio extracted in X.Xs
```

If the video has no audio track, log a clear warning and set transcription to an empty string — **do not crash**.

Step 2 — Transcribe with `distil-whisper/distil-large-v3`

Use the `faster-whisper` library with the `distil-large-v3` model.

- On **GPU**: use `compute_type="float16"` and `device="cuda"`
- On **CPU**: use `compute_type="int8"` and `device="cpu"`
- Model files are cached locally after first download. Check the cache before loading.

```
LOG: [TRANSCRIBE] Loading distil-large-v3 (faster-whisper)...
LOG: [TRANSCRIBE] Device: CPU | Compute: int8
LOG: [TRANSCRIBE] Starting transcription...
LOG: [TRANSCRIBE] Progress: segment 1/N — "Hello and welcome to..."
LOG: [TRANSCRIBE] Progress: segment 2/N — "Today we're going to..."
```

```
...
LOG: [TRANSCRIBE] ✓ Transcription complete — X segments, ~X words, X.Xs elapsed
LOG: [TRANSCRIBE] Writing → /tmp/videoanalyzer_<uuid>/transcription.txt
```

The transcription file must include segment timestamps:

```
[00:00:00 → 00:00:05] Hello and welcome to this tutorial.
[00:00:05 → 00:00:12] Today we're going to cover the basics of Python.
```

4.3 Frame Extraction (Scene-Change Based)

File: `video/frame_extractor.py`

This module is directly based on the existing scene-change extraction script (see §10). No new logic needs to be invented here — it is a direct integration of that script into the pipeline with the following additions:

GPU Batch Mode vs CPU Sequential Mode

- **CPU mode**: Extract and save frames one at a time (existing behavior).
- **GPU mode**: Batch-decode frames using OpenCV's CUDA-accelerated backend where available, then process the batch for scene-change comparison. This accelerates the *extraction* step only — the actual scene-change comparison math (histogram + SSIM) is the same on both.

```
LOG: [FRAMES] Starting keyframe extraction...
LOG: [FRAMES] Mode: GPU-BATCH | Video: 1280x720 @ 30fps
LOG: [FRAMES] Thresholds → hist=0.28, ssim=0.89, min_interval=8 frames
LOG: [FRAMES] Processing frame 0 → saved keyframe_0000 (first frame)
LOG: [FRAMES] Frame 142 → SCENE CHANGE [hist=0.341] → saved keyframe_0001
LOG: [FRAMES] Frame 287 → SCENE CHANGE [ssim=0.812] → saved keyframe_0002
...
LOG: [FRAMES] ✓ Extraction complete — X keyframes from X total frames (X.Xs)
LOG: [FRAMES] Frames saved → /tmp/videoanalyzer_<uuid>/frames/
```

Frames are saved into the temp directory for this run, not `artifacts/video_frames/`, since they will be cleaned up after analysis.

4.4 Frame Analysis with Qwen3-VL:2b

File: `video/frame_analyzer.py`

Each extracted keyframe is sent to the `qwen3-vl:2b` model running via Ollama's local API (`http://localhost:11434/api/generate`).

Analysis prompt per frame

Send each frame as a base64-encoded image with the following system instruction:

```
You are a precise visual analyst. Describe this video frame in exhaustive detail.
Include: all visible objects, text, people (appearance, expressions, actions),
environment, colors, spatial layout, any on-screen graphics or UI elements,
and what is happening in the scene. Be thorough — your description will be
used to reconstruct a full understanding of this video.
```

Processing order

- Frames are analyzed **sequentially** (one at a time) regardless of GPU/CPU. Vision model inference is a single-model bottleneck — batching does not apply here.
- Each frame description is immediately appended to `frame_analyses.txt` in the temp directory as it completes, so progress is preserved incrementally.

Output format per frame

```
=== KEYFRAME 0001 | Frame #142 | Timestamp ~00:00:04 ===
[Full LLM description here]

=== KEYFRAME 0002 | Frame #287 | Timestamp ~00:00:09 ===
[Full LLM description here]
```

Logs

```
LOG: [VISION] Analyzing X keyframes with qwen3-vl:2b...
LOG: [VISION] Frame 0001/X (frame #142, ~00:00:04)...
LOG: [VISION] ✓ Frame 0001 described (X tokens, X.Xs)
LOG: [VISION] Frame 0002/X (frame #287, ~00:00:09)...
LOG: [VISION] ✓ Frame 0002 described (X tokens, X.Xs)
```

```
...
LOG: [VISION] ✓ All frames analyzed — X.Xs total
LOG: [VISION] Writing → /tmp/videoanalyzer_<uuid>/frame_analyses.txt
```

4.5 Master Summary Generation

File: `llm/summarizer.py`

This is the most important step. The LLM (`qwen3.5:9b` via Ollama) receives the full transcription and all frame descriptions and is instructed to produce an extraordinarily detailed summary.

System prompt

```
You are a senior multimedia analyst producing a comprehensive written record of a video.
You have been provided:
  1. A full timestamped transcription of all spoken audio in the video.
  2. Detailed visual descriptions of every significant scene/frame in the video.

Your task: Synthesize both sources into a single, exhaustive, multi-page written analysis.

Structure your summary as follows:
  - OVERVIEW: What is this video? What is its purpose, genre, and main subject?
  - DETAILED NARRATIVE: Walk through the video chronologically, integrating what was
    said and what was shown at each point. Do not summarize loosely — describe everything
    in precise, specific language. This section should be very long.
  - KEY POINTS & CONCEPTS: A thorough list of every important idea, instruction, claim,
    or demonstration present in the video.
  - VISUAL HIGHLIGHTS: Notable visual elements, UI shown, graphics, on-screen text,
    demonstrations.
  - SPEAKERS & PARTICIPANTS: Who appears, what they say, their apparent role.
  - TONE & STYLE: Pacing, presentation style, intended audience.

Do not truncate. Do not summarize loosely. A 30-second video should produce at minimum
600–800 words of analysis. A 5-minute video should produce 3,000–5,000 words.
Be precise, be thorough, and leave nothing out.
```

Logs

```
LOG: [SUMMARY] Sending transcription + X frame descriptions to qwen3.5:9b...
```

```
LOG: [SUMMARY] Input size: ~X tokens
LOG: [SUMMARY] Generating master summary (this may take a moment)...
LOG: [SUMMARY] ✓ Summary generated — X words, X tokens (X.Xs)
```

4.6 Prompt Q&A Layer (Optional)

File: `llm/summarizer.py` → `answer_with_summary()`

This step only runs if the user provided a `prompt` argument. It takes the master summary produced in §4.5 and the user's prompt, and asks the LLM to answer the prompt using the summary as the sole knowledge source.

System prompt

```
You are an expert analyst. You have access to a detailed summary of a video (provided below).
A user has asked a specific question about this video.

Your task:
  1. Answer the user's question precisely and completely.
  2. After your answer, cite the relevant sections of the summary that support your answer,
     quoting or paraphrasing key lines to show your reasoning.

Do not go beyond the summary. If the answer cannot be found in the summary, say so clearly.
```

User message format

```
SUMMARY:
{master_summary}

USER QUESTION:
{prompt}
```

Output structure returned to caller

```
ANSWER:
{direct_answer_to_prompt}
```

RELEVANT CONTEXT FROM SUMMARY:
{cited_sections_from_summary}
```

#### Logs

```
LOG: [QA] User prompt detected: "{prompt[:60]}..."
LOG: [QA] Sending summary + prompt to qwen3.5:9b...
LOG: [QA] ✓ Prompt answer generated — X words (X.Xs)
```

4.7 Output & Cleanup

File: `utils/cleanup.py`

Output

The `analyze()` function returns an `AnalysisResult` dataclass:

```python
@dataclass
class AnalysisResult:
    summary: str                    # Always present — full master summary
    prompt_answer: str | None       # Present only if prompt was provided
    keyframe_count: int             # Number of keyframes analyzed
    transcription: str              # Full timestamped transcription
    duration_seconds: float         # Total pipeline wall-clock time
    video_path: str                 # Echo of input path
```

The summary is also printed to terminal in a clean, readable format.

Cleanup

After returning the result, the temp directory (and all its contents — audio, frames, transcription, frame analysis files) is deleted automatically.

```
LOG: [CLEANUP] Removing temp directory: /tmp/videoanalyzer_<uuid>/
LOG: [CLEANUP] ✓ Cleaned up X files
LOG: [DONE] ✨ Total pipeline time: X.Xs
```
```

---

5. Models & Dependencies

5.1 Python dependencies (`requirements.txt`)

```
faster-whisper          # Whisper transcription (distil-large-v3)
opencv-python           # Frame extraction
scikit-image            # SSIM computation
numpy                   # Array ops
requests                # Ollama API calls
Pillow                  # Image encoding (base64 for vision model)
torch                   # GPU detection and tensor ops
ffmpeg-python           # Audio extraction wrapper
```

> **Note:** `ffmpeg` binary must also be installed on the system. The pipeline should detect its presence on startup and log a clear error with install instructions if missing.

5.2 Whisper model management

- Model: `distil-whisper/distil-large-v3`
- Managed by `faster-whisper` — cached to `~/.cache/huggingface/` automatically after first download.
- On startup, `model_manager.py` checks for the cached model. If absent, it triggers a download with a clear log:

```
LOG: [MODEL] distil-large-v3 not found in cache.
LOG: [MODEL] Downloading distil-large-v3... (this is a one-time download, ~1.5GB)
LOG: [MODEL] ✓ Model ready.
```

5.3 Ollama model management & process control

- Models needed: `qwen3-vl:2b` (vision), `qwen3.5:9b` (language)
- Managed by `llm/ollama_manager.py`

Ollama process lifecycle

On every run, the pipeline must:

1. Check if Ollama is already running by hitting `GET http://localhost:11434/api/tags`.

2. If not running → call `subprocess.Popen(["ollama", "serve"])`, wait up to 10 seconds for it to respond, then proceed.
3. If Ollama binary is not installed → log a clear error with installation instructions and raise `RuntimeError`.

```
LOG: [OLLAMA] Checking if Ollama is running...
LOG: [OLLAMA] Not running — starting ollama serve...
LOG: [OLLAMA] ✓ Ollama is live at http://localhost:11434
```

Model availability check

After confirming Ollama is running, call `GET /api/tags` and check the returned model list.

For each required model (`qwen3-vl:2b`, `qwen3.5:9b`):
- If present → log `✓ Model qwen3-vl:2b found.`
- If absent → trigger `ollama pull <model>` via subprocess and stream the download progress to the log.

```
LOG: [OLLAMA] Required models: qwen3-vl:2b, qwen3.5:9b
LOG: [OLLAMA] ✓ qwen3-vl:2b — found
LOG: [OLLAMA] × qwen3.5:9b — not found
LOG: [OLLAMA] Pulling qwen3.5:9b (one-time download)...
LOG: [OLLAMA] Pull progress: 12% | 23% | 45% | 78% | 100%
LOG: [OLLAMA] ✓ qwen3.5:9b ready.
```

---

6. Hardware Detection & Optimization

File: `utils/hardware.py`

On startup, the pipeline runs a full hardware survey and logs the result. This determines behavior throughout the pipeline.

Detection logic

```python
import torch

def detect_hardware() -> HardwareConfig:
    if torch.cuda.is_available():
        device = "cuda"
```

```
        gpu_name = torch.cuda.get_device_name(0)
        vram_gb = torch.cuda.get_device_properties(0).total_memory / 1e9
    else:
        device = "cpu"
        gpu_name = None
        vram_gb = 0

    return HardwareConfig(device=device, gpu_name=gpu_name, vram_gb=vram_gb)
```

Log output

```
LOG: [HARDWARE] ── Hardware Survey ──────────────────────────
LOG: [HARDWARE] Device:       CUDA (GPU)
LOG: [HARDWARE] GPU:          NVIDIA GeForce RTX 4090
LOG: [HARDWARE] VRAM:         24.0 GB
LOG: [HARDWARE] Torch dtype:  bfloat16
LOG: [HARDWARE] ──────────────────────────────────────────────
```

Or for CPU:

```
LOG: [HARDWARE] Device:       CPU
LOG: [HARDWARE] Whisper dtype: int8
```

GPU tier optimization table

GPU Class	Examples	torch_dtype	Frame Extraction	Whisper
High-end (≥20GB VRAM)	A100, H100, RTX 4090	`bfloat16`	GPU batch mode	float16
Mid-range (8–20GB VRAM)	RTX 3080, RTX 4070	`float16`	GPU batch mode	float16
Low-end GPU (<8GB VRAM)	RTX 3060, GTX 1080	`float16`	Sequential	float16
CPU (no GPU)	Any	`int8`	Sequential	int8

The `HardwareConfig` object is passed to every pipeline module that needs it. No module should do its own hardware detection — they all receive it from the shared config.

---

7. Module API (Importable Interface)

File: `video_analyzer/__init__.py`

The entire pipeline is exposed as a single importable function:

```python
from video_analyzer import analyze

result = analyze(
    video_path="ingestion/my_video.mp4",
    prompt="What products are demonstrated in this video?"
)

print(result.summary)          # Full multi-page summary
print(result.prompt_answer)    # Answer to the question
print(result.transcription)    # Full timestamped transcript
print(result.keyframe_count)   # e.g. 14
print(result.duration_seconds) # e.g. 47.3
```

`AnalysisResult` dataclass

```python
@dataclass
class AnalysisResult:
    summary: str
    prompt_answer: str | None
    keyframe_count: int
    transcription: str
    duration_seconds: float
    video_path: str
```

Terminal output behavior

Even when used as a module (not run directly), the pipeline always prints structured log output to `stdout`. This is intentional — logs are a first-class feature of this system, not optional debug output. If a caller wants to suppress them, they can redirect `stdout`.

---

8. Logging Standard

File: `utils/logger.py`

Every single log line must follow this format:

```
[TIMESTAMP] [MODULE] MESSAGE
```

Example:

```
[00:00:01.234] [AUDIO]      Extracting audio from video...
[00:00:03.891] [AUDIO]      ✓ Audio extracted in 2.7s
[00:00:03.892] [TRANSCRIBE] Loading distil-large-v3...
[00:00:07.441] [TRANSCRIBE] Device: CPU | Compute: int8
[00:00:07.442] [TRANSCRIBE] Starting transcription...
[00:00:08.011] [TRANSCRIBE] Segment 1/12 — "Hello and welcome..."
[00:00:14.220] [TRANSCRIBE] ✓ Transcription complete — 12 segments, 143 words, 6.8s
[00:00:14.221] [FRAMES]     Starting keyframe extraction...
[00:00:14.880] [FRAMES]     Frame 0 → saved (first frame)
[00:00:16.100] [FRAMES]     Frame 142 → SCENE CHANGE [hist=0.341] → saved
...
```

Logging rules

1. Every function entry of significance gets an opening log line.
2. Every function completion gets a `✓` success log with timing.
3. All errors log the full exception message before raising.
4. Use a single centralized logger — not `print()` scattered everywhere.
5. Module tags are fixed-width (padded to 12 chars) for visual alignment.
6. Timestamps are relative to pipeline start, not wall clock.

---

9. Configuration Reference

File: `video_analyzer/config.py`

All tunable parameters live here. No magic numbers anywhere else in the codebase.

```python
# ── Frame Extraction ──────────────────────────────
HIST_THRESHOLD       = 0.28    # χ² histogram distance. Lower = more sensitive.
SSIM_THRESHOLD       = 0.89    # Structural similarity. Lower = more keyframes.
MIN_FRAME_INTERVAL   = 8       # Minimum frames between two saved keyframes.

# ── Models ────────────────────────────────────────
WHISPER_MODEL        = "distil-large-v3"
```

```
VISION_MODEL         = "qwen3-vl:2b"
SUMMARY_MODEL        = "qwen3.5:9b"

# ── Ollama ──────────────────────────────────────────
OLLAMA_HOST          = "http://localhost:11434"
OLLAMA_STARTUP_WAIT  = 10      # Seconds to wait after starting ollama serve

# ── Output ──────────────────────────────────────────
PRINT_SUMMARY        = True    # Print summary to terminal on completion
TEMP_DIR_PREFIX      = "/tmp/videoanalyzer_"

# ── Vision prompt ───────────────────────────────────
FRAME_ANALYSIS_PROMPT = """
You are a precise visual analyst. Describe this video frame in exhaustive detail.
Include: all visible objects, text, people (appearance, expressions, actions),
environment, colors, spatial layout, any on-screen graphics or UI elements,
and what is happening in the scene. Be thorough.
"""

# ── Summary prompt ──────────────────────────────────
SUMMARY_SYSTEM_PROMPT = """
You are a senior multimedia analyst producing a comprehensive written record of a video.
[...full prompt as specified in §4.5...]
"""

# ── Q&A prompt ──────────────────────────────────────
QA_SYSTEM_PROMPT = """
You are an expert analyst with access to a detailed video summary.
[...full prompt as specified in §4.6...]
"""
```

---

10. Frame Extraction Module (Reference Code)

This is the existing scene-change extraction script that `video/frame_extractor.py` must be built from. ***Do not rewrite the core logic** — integrate it directly and extend it with GPU batch support and the logging standard.

```python
import cv2
import os
import numpy as np
from skimage.metrics import structural_similarity as ssim
```

```python


def extract_frames(
    video_path,
    output_folder,
    hist_threshold=0.30,
    ssim_threshold=0.88,
    min_frame_interval=8,
    use_grayscale_for_ssim=True,
    hist_method=cv2.HISTCMP_CHISQR
):
    """
    Extract keyframes on structural / scene changes.
    Two checks:
      1. Color histogram difference
      2. SSIM structural similarity
    Triggers save when either difference is large enough.
    """
    if not os.path.exists(output_folder):
        os.makedirs(output_folder)

    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        print(f"Error opening video: {video_path}")
        return

    fps = cap.get(cv2.CAP_PROP_FPS) or 30
    ret, prev_frame = cap.read()
    if not ret:
        return

    prev_hist = []
    for channel in cv2.split(prev_frame):
        h = cv2.calcHist([channel], [0], None, [256], [0, 256])
        h = cv2.normalize(h, h).flatten()
        prev_hist.append(h)

    prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)

    frame_idx = 0
    saved_count = 0
    cv2.imwrite(
        os.path.join(output_folder, f"keyframe_{saved_count:04d}_frame_{frame_idx:06d}.jpg"),
        prev_frame
    )
```

```
    saved_count += 1
    last_save_idx = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frame_idx += 1

        if frame_idx - last_save_idx < min_frame_interval:
            continue

        hist_diff_max = 0
        for i, channel in enumerate(cv2.split(frame)):
            h = cv2.calcHist([channel], [0], None, [256], [0, 256])
            h = cv2.normalize(h, h).flatten()
            diff = cv2.compareHist(prev_hist[i], h, hist_method)
            hist_diff_max = max(hist_diff_max, diff)

        curr_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        ssim_value = ssim(
            prev_gray, curr_gray,
            data_range=curr_gray.max() - curr_gray.min()
        )

        should_save = False
        reason = ""
        if hist_diff_max > hist_threshold:
            should_save = True
            reason += f"hist={hist_diff_max:.3f} "
        if ssim_value < ssim_threshold:
            should_save = True
            reason += f"ssim={ssim_value:.3f}"

        if should_save:
            filename = os.path.join(
                output_folder,
                f"keyframe_{saved_count:04d}_frame_{frame_idx:06d}.jpg"
            )
            cv2.imwrite(filename, frame)
            # Update references
            prev_hist = []
            for channel in cv2.split(frame):
                h = cv2.calcHist([channel], [0], None, [256], [0, 256])
                h = cv2.normalize(h, h).flatten()
```

```
                prev_hist.append(h)
            prev_gray = curr_gray
            last_save_idx = frame_idx
            saved_count += 1

    cap.release()
```

When integrating this into the pipeline:
- Replace all `print()` calls with the centralized logger.
- Accept `HardwareConfig` as a parameter and enable GPU batch mode when `device == "cuda"`.
- Save frames to the run's temp directory, not a fixed output folder.
- Return the list of saved frame paths + their approximate timestamps for use in §4.4.

---

11. Development Guidelines

Code documentation standard

Every file must start with a module-level docstring explaining its purpose, inputs, and outputs. Every function must have a docstring. Every non-obvious logic block must have an inline comment. The test for "non-obvious" is simple: if you had to think about it for more than 5 seconds, it needs a comment.

```python
# ── Bad ──────────────────────────────────
ssim_value = ssim(prev_gray, curr_gray, data_range=255)

# ── Good ─────────────────────────────────
# SSIM measures structural similarity between two grayscale frames.
# A value close to 1.0 means nearly identical frames.
# We save a keyframe when this drops below ssim_threshold,
# indicating a meaningful scene change has occurred.
ssim_value = ssim(prev_gray, curr_gray, data_range=255)
```

Module independence

Each module (`audio/`, `video/`, `llm/`, `utils/`) must be independently importable and testable. No module should import from a sibling module. All cross-module communication goes through `analyzer.py`.

No magic numbers

Every numeric constant — thresholds, wait times, token limits, model names — lives in `config.py`. If you write a literal number in a module file, it is a bug.

One-time downloads only

Model downloads (Whisper, Ollama models) must be gated by a cache check. The check must happen before any download is attempted, and the result must be logged. On re-runs, the log should confirm the cached model is being used, not re-downloaded.

Fail loudly and clearly

If a required dependency is missing (ffmpeg, ollama binary, Python package), do not raise a cryptic `AttributeError` deep in the stack. Catch it at startup, log a specific human-readable message like `"ffmpeg binary not found. Install with: brew install ffmpeg (macOS) or apt install ffmpeg (Ubuntu)"`, and raise a `RuntimeError` immediately.

---

12. Error Handling Strategy

Scenario	Behavior
Video file not found	`FileNotFoundError` with full path in message
Video has no audio	Log warning, continue with empty transcription
ffmpeg not installed	`RuntimeError` with install instructions
Ollama not installed	`RuntimeError` with install instructions
Ollama model pull fails	Retry once, then raise `RuntimeError`
Frame extraction produces 0 frames	Log warning, skip vision step, summarize from transcription only
Vision model returns empty response	Log warning for that frame, use placeholder text, continue
LLM summary generation fails	Raise `RuntimeError` — this is unrecoverable
Temp directory cleanup fails	Log warning only — do not raise, the result is already returned

---

End of specification.